



gRPC: 5 Years Later

A real-world engineering journey.

Let's explore its benefits, challenges, and practical applications.



by Konstantin Ostrovsky



KubeCon



CloudNativeCon

Europe 2025

torq=

Konstantin Ostrovsky



Chief Architect at Torq.io



Backend Engineer

Low level to high level.



I Love Cybersecurity Startups



Passionate about optimizing CI build times



Blogging at kostyay.com

Mostly on backend development and infrastructure engineering



Torq Tech Stack

Security Automation

In house workflow orchestration engine executing many of millions of automations a day

Service Architecture

Microservices written in Go with gRPC for all communication, powering both backend services and frontend interfaces

Data Management

PostgreSQL for structured data, Redis for caching and real-time operations

Cloud Infrastructure

Heavy users of Google Cloud Storage, Pubsub for messaging queues, Cloud Tracing for Observability

Why did we pick gRPC?

Mainly for the developer experience..

1

Code Generation

Quality client and server generation backed by a large community

2

Backward compatibility

Out of the box BC, by design

3

Networking Performance

Binary protocol, efficient data transfer.



What is gRPC?

gRPC is **Google Remote Procedure Call**.

It is a high-performance, open-source RPC framework.

Language Agnostic

Works with many languages.

Protocol Buffers

Uses protocol buffers by default.

Open Source (CNCF)

Community-driven development.

Quick Start

1

Define

Define the service using .proto files.

2

Generate

Generate server and client code.

3

Implement

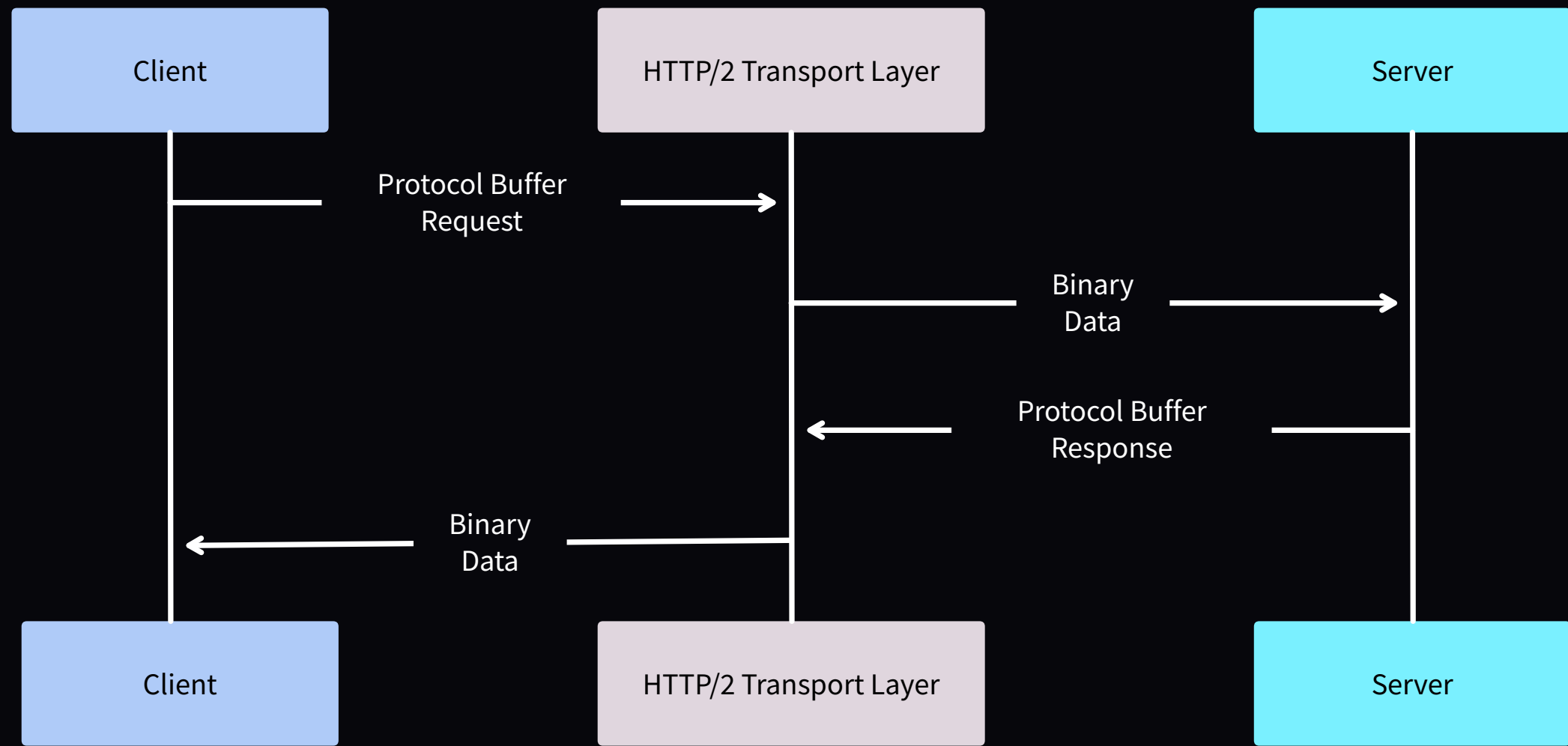
Implement the service logic.

4

Communicate

Clients communicate with the server.

Typical Data Flow



Key Technical Advantages

1 Binary protocol over HTTP/2

Provides significantly better performance than traditional REST APIs.

2 Backward compatibility by design

Protocol Buffers enforce clear rules for evolving APIs without breaking existing clients.

3 Strongly typed API contract

Using .proto files to eliminate ambiguity and ensure consistency.

4 Rich language generation support

Robust and well-maintained tooling across multiple languages.

5 Extensibility by design

Allows building additional functionality on top as needed.

Getting practical



Ensuring working APIs



Lint everything

Enforced strict coding standards across all services. Ensure consistent naming, structure, and documentation.



Backward Compatibility Verification

Integrated into CI/CD pipeline. Prevents accidental breaking changes to existing APIs.



Limited Message Sharing

Reduced dependencies between services. Improves isolation and prevents tight coupling.



Custom Developer Tooling

Low friction company wide docker container for the developers, with everything baked in.

Repository level implementation



Multi-Repo Architecture

We distribute services across multiple repositories for better isolation and business ownership boundaries.



Service-Aligned APIs

Proto definitions live alongside their implementing services. This creates clear accountability and encourages reduction of cross-repo dependencies.



Go-Based Client Libraries

We use go, API clients are consumed as Go modules.



Developer Workflow

Generated code is committed to Git repositories.

Typical repo

1 API in service Repo

Proto files in the **api** directory, by service and version.

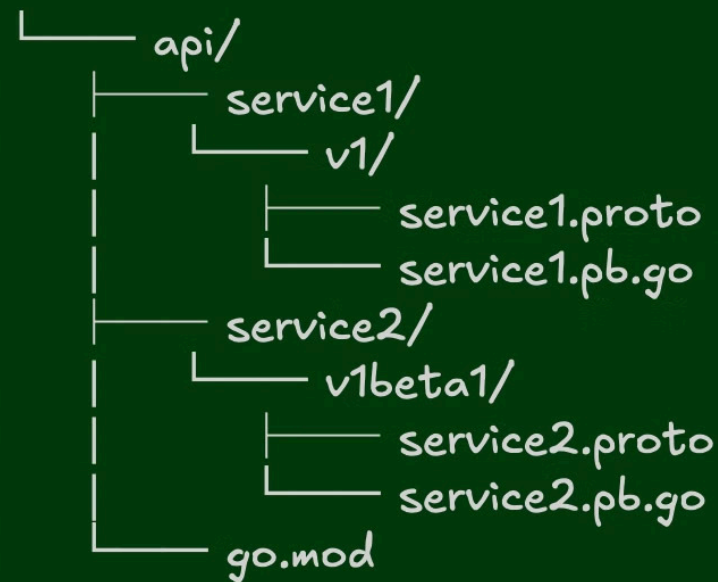
2 Standalone Module

The **api** directory is a reusable Go module.

3 Cross-Platform Support

Publish npm packages for TypeScript clients for the Front-end application.

Service Directory Layout



Service

The Benefits gRPC Gives Us

Unified API Standards

Consistent interfaces across services. Developers know what to expect from every API.

Developer Mobility

Engineers move between codebases with ease. Familiar patterns reduce ramp-up time. Same code everywhere.

Less Code To Review

We trust the generated Clients code

Extensions

Use extensions to annotate services with AuthZ/AuthN at the API level.

Middleware

We've build a lot of middleware that integrates with gRPC (Auth/O11y/Metrics/Flags)

gRPC Extensibility

```
1 service ExampleService {
2   rpc VibeCode (VibeCodeRequest) returns (VibeCodeResponse) {
3     option (torq.rbac.v1.authz) = {scopes: [SCOPE_VIBE_CODE]};
4   }
5 }
6
7 message VibeCodeRequest {
8   string prompt = 1 [(buf.validate.field).string.min_len = 1];
9 }
10
11 message VibeCodeResponse {
12   string code = 1;
13 }
14
```

Lessons Learned



Challenges using gRPC

Implementation

Directory structure, dependency management is unclear

Tooling

Standard tools lack great developer experience

Frontend

Not a first-class citizen, gRPC-Web is quite odd.

Not ideal for Customer Facing APIs

Your customers will likely prefer using REST API for your APIs



Kubernetes Load Balancing

Kubernetes doesn't load balance HTTP/2 connections well

1 gRPC uses HTTP/2

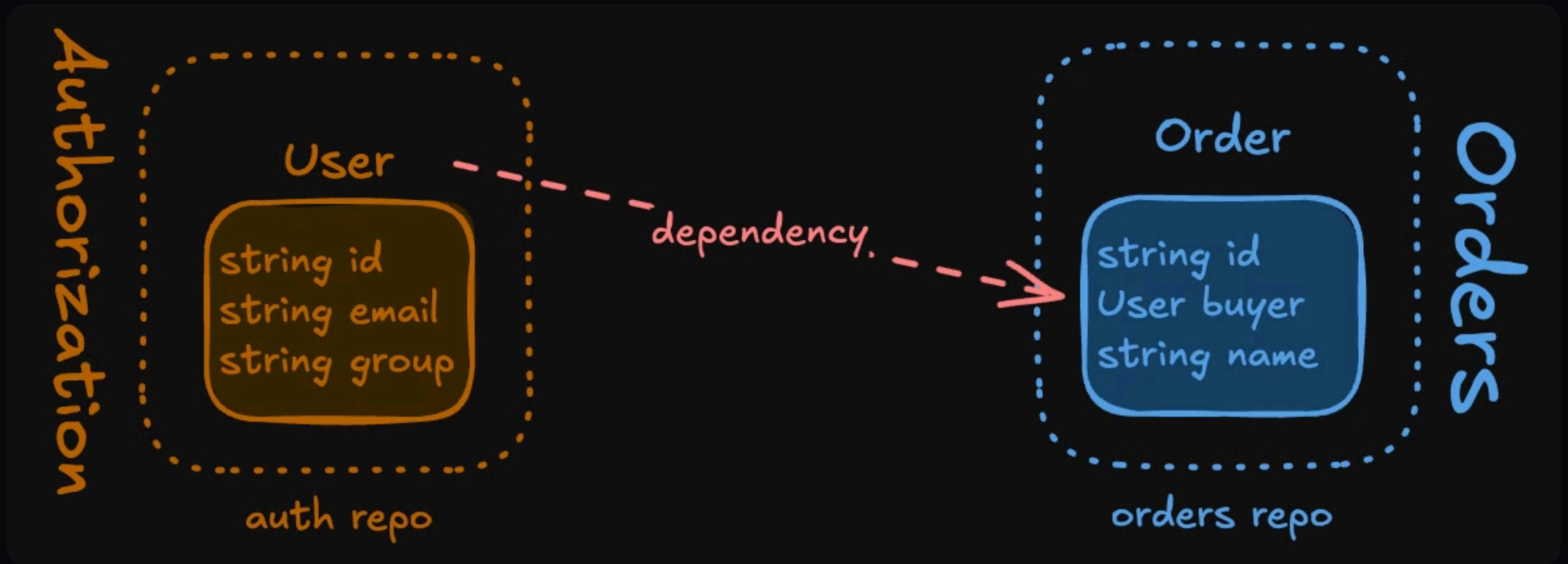
Every connection is a separate stream in the **same** TCP/IP connection

2 Kubernetes LB is L4

This works fine for HTTP1/1.1 but not for HTTP2

3 Service Mesh or Headless Service needed

Avoid reusing messages across services



gRPC Web: Browser Challenges



Developer Experience

More friction than REST API



Debugging

Requires special browser extension



Caching

Uses POST requests, bypassing browser's

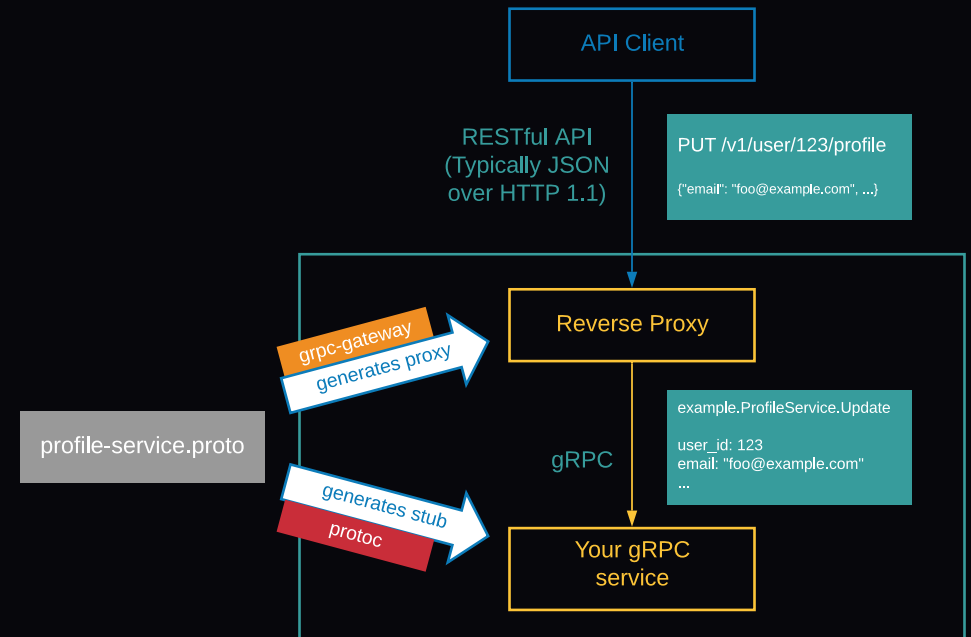


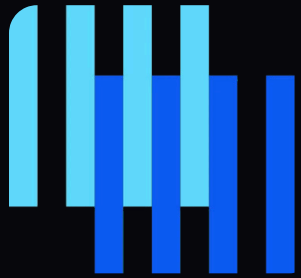
Proxy layer

Needs translation between gRPC-Web and gRPC

gRPC is not ideal for external public APIs

- The gRPC-Gateway is a plugin of the Google protocol buffers compiler **protoc**.
- Generates a reverse-proxy server which translates a RESTful HTTP API into gRPC.
- This server is generated according to the **google.api.http** annotations in your service definitions.
- This helps you provide your APIs in both gRPC and RESTful style at the same time.





Modern tools make the
difference



buf: gRPC Tooling Simplified



Complete Protobuf Ecosystem

One unified toolchain manages the entire protocol buffer workflow seamlessly.



Dependency management

Pain free cross module dependency management



Plugins

Say goodbye to plugin and compiler version mismatches across teams.



Built-in Linting & Schema Management

Enforce standards with integrated linting and centralized schema control.

before

```
> protoc -I /api \  
  --go_out=plugins=grpc,paths=source_relative:/goclient \  
  --js_out=import_style=commonjs:/jsclient \  
  --grpc-web_out=import_style=commonjs,mode=grpcwebtext:/jsclient \  
  /api/time/v1/time_service.proto
```

after

```
> buf generate
```




```
curl \
  --header "Content-Type: application/json" \
  --data '{"sentence": "I feel happy."}' \
  https://demo.connectrpc.com/connectrpc.eliza.v1.ElizaService/Say
```

Connect: The Effortless Path to Powerful HTTP APIs

- Implements its own JSON-based protocol for gRPC that fully supports HTTP caching mechanisms
- Seamlessly integrates with both gRPC and gRPC-Web as underlying transport protocols in addition to its own new protocol
- Delivers significantly improved code generators optimized for browser-based programming languages

So was it all worth it?



Absolutely, standardization is great.



Requirements were met:

- Great developer experience
- Unified language defining clients and servers
- Decent generated code



New tooling makes everything much simpler



Questions?



You can find a more detailed breakdown in my blog post.

